

NLP Project Report

Tanay Shailesh Dubey, t.dubey@oth-aw.de

1 Introduction

This report discusses a solution to the given Natural Language Processing (NLP) problem statement. The objective is to compile a small, structured dataset that contains images of quantum circuits, their corresponding descriptive text, along with some additional relevant metadata. This dataset is generated by automatically extracting information from the given list of research papers based on quantum physics domain. To achieve this, the solution uses various NLP techniques to identify, process, and organize meaningful textual content from the papers. The overall approach is designed with the final goal in mind: preparing a high-quality dataset suitable for training an Image-to-Text model.

2 Data Extraction

The data extraction pipeline used in this solution begins by fetching the PDF sources of the research papers. Once a paper is fetched, the pipeline processes the document page by page. On each page, it identifies image captions and extracts them as textual information. For every detected caption, the system captures the corresponding image located above the caption and saves both the image and its caption. Also, for each extracted image, the pipeline searches through the entire paper to collect related descriptive text and other relevant metadata, ensuring that each image is paired with meaningful contextual information. The source code file `main.py`, which actually is the final solution source code, also showcases the data extraction pipeline.

2.1 PDF-Based Paper Retrieval

The data extraction pipeline begins by reading a list of arXiv paper IDs from the file `paper_list_71`. For each paper ID, it fetches the corresponding PDF directly from the arXiv.org website. Although the problem statement allows access to LaTeX source files, this solution refrains from using them and relies solely on PDFs for the following reasons:

1. A small but significant number of papers do not provide LaTeX source files, while every paper is available in PDF format. Using PDFs ensures that no relevant papers or images are excluded from the dataset.
2. Extracting accurate page numbers for images is not possible using solely the LaTeX source files, whereas PDFs preserve this information.
3. Many quantum circuits in these papers are created using vector graphics. Such images are not available in standard image formats, would require compiling the full LaTeX source using external, non-Python tools, increasing system dependency and setup complexity.
4. Using a combination of LaTeX-based and PDF-based extraction methods would significantly increase the complexity of the pipeline and make it more prone to errors.

2.2 PDF Processing and In-Memory Handling

The function `download_pdf_to_bytes()` in `main.py` is responsible for fetching the PDF of a given arXiv paper and returning it as a BytesIO object for in-memory processing. This allows the processing of each PDF in the system's RAM itself, avoiding unnecessary use of disk storage to store a large number of PDF files.

The main loop maintains certain variables to track the number of papers processed and images extracted. For handling and analyzing the PDF content, the solution uses the PyMuPDF library. This library allows efficient processing of the in-memory PDF objects and is responsible for extracting relevant information such as images, captions, and page-level data from each paper.

2.3 Initial Data Extraction from PDFs

The PDF processing begins by extracting all of the textual data from the PDF file. The contents of the PDF is divided into text blocks by PyMuPDF's `get_text()` function. The code iterates through each page of the PDF and applies the `get_text("blocks")` method to capture all text blocks present on that page. For every extracted text block, the system records its bounding boxes coordinates, starting and ending character indices, and the corresponding page number. This initial pass over the entire PDF creates a "coordinate-aware map" of the document, which serves as a foundation for accurately locating images, captions, and their related textual information in subsequent processing steps.

2.4 Second Iteration: Extracting Image Captions

The system iterates over the PDF pages one more time. In the second pass over the PDF, the system focuses on identifying image captions. For each page, it retrieves the text blocks and applies a regular expression (`CAPTION_REGEX` in `regex_patterns.py`) to detect those text blocks that contain captions. Observations show that image captions typically start with "Figure", "Fig", or similar variations, and are often followed by punctuation such as ".", ":", or "-". The `CAPTION_REGEX` is designed using this pattern to accurately extract captions from the text.

Each extracted caption serves as an important piece of descriptive text, which forms part of the metadata in the final dataset. Simultaneously, the figure number is also captured from the caption, usually appearing immediately after "Figure" or "Fig". This figure number and the page number become another key metadata elements for our dataset.

2.5 Extracting Remaining Descriptive Texts and Their Positions

Image captions alone do not provide all the descriptive information about an image. Often, a figure is referenced elsewhere in the paper, and the surrounding text provides additional context or explanation related to the image. This text can also be considered descriptive and is included in the dataset metadata.

The `extract_descriptions_with_pos()` in `main.py` function handles this process. It uses the text blocks collected during the initial pass to search the entire paper, since mentions of a figure can appear on any page. A regular expression is used to locate mentions of the figure using the word "Figure" (or its variations) along with the corresponding figure number. Once all relevant text blocks are identified, the block containing the caption is removed, as it has already been extracted.

From the remaining blocks, the system extracts up to five sentences before and after the figure mention to capture meaningful descriptive context. For each piece of descriptive text, as well as for the caption, the start and end character positions are calculated. These represent the indices of the characters, when the entire text from the entire pdf is compiled into a single string. These indices allow precise retrieval of the descriptive text and are also included as important metadata in the final dataset.

Thus the descriptions field in `dataset_71.json`, include the text surrounding the image mentions, as well as the image captions.

2.6 Image Extraction

The images in research papers can generally be divided into two types: rasterized images (e.g., .png, .jpg) and vector-based images (e.g., .svg, .eps). Vector-based images cannot be directly extracted from PDFs because these images are generated from code, while rasterized images can be extracted using standard PDF functions. However, directly extracting raster images and mapping them to their captions is challenging and is very prone to errors. There are always chances of mismatch between figure and its caption.

To address this, the solution relies on a spatial relationship between captions and images. It has been observed that, in majority of the cases, an image is located directly above its caption. The `crop_and_rasterize()` in `main.py` function leverages this by locating each caption and defining a rectangular region above it that is likely to contain the corresponding image. The function identifies the nearest "long text" or full sentence above the caption which will act as a top boundary for the image, preventing body text from being included. A crop rectangle is then created, slightly extended with margins for safety, and this region is rendered at 300 DPI using PyMuPDF and saved as a .png file.

While this method successfully extracts most images, some limitations exist. Rarely, images appear to the left or right of their captions, as shown in Figure 1(a), which the system ignores to avoid errors. In certain cases, the top or side boundary may not be accurately recognized, resulting in clipped images, as shown in Figure 1(b), or inclusion of surrounding text, as shown in Figure 2. Therefore, an image is considered a True Positive only if it clearly shows the significant parts of the quantum circuit.

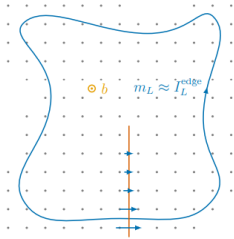


Figure 1: Pictorial representation of our main results: For locally interacting fermions on a two-dimensional lattice with perpendicular magnetic field b , satisfying local indistinguishability (see details in section 2.1) at positive temperature, the edge current I_L^{edge} , which is the bond current through the vertical line, is localized near the boundary and approximately equals the magnetization m_L (Theorem I). The latter is a bulk quantity, i.e. it converges in the thermodynamic limit $L \rightarrow \infty$ and does not depend on the details near the boundary (Theorem II). This independence allows for the rough edges in the picture.

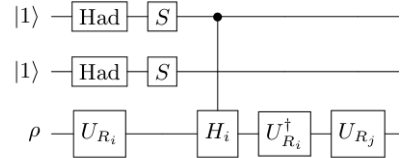


Figure 1: (a) Caption on the right of the image and (b) Image clipped from the right.

- 1: Prepare the squeezed vacuum state $|\Psi_{2^{-n}\Delta}\rangle \leftarrow \mathcal{S}(n \log 2 + \log 1/\Delta) |\text{vac}\rangle$.
 - 2: Prepare the qubit state $|+\rangle$. Denote the resulting state by $|\Phi^{(0)}\rangle \leftarrow |\Psi_{2^{-n}\Delta}\rangle \otimes |+\rangle$.
 - 3: Apply $(e^{iP} \otimes I)V$ to the state $|\Phi^{(0)}\rangle$ yielding $|\Phi^{(1)}\rangle \leftarrow (e^{iP} \otimes I)V |\Phi^{(0)}\rangle$.
 - 4: **for** $k \in \{2, \dots, n\}$ **do**
 - 5: Apply V to the state $|\Phi^{(k-1)}\rangle$, yielding $|\Phi^{(k)}\rangle \leftarrow V |\Phi^{(k-1)}\rangle$.
 - 6: **return** the first register of the state $|\Phi^{(n)}\rangle$, i.e., the state $\text{tr}_{\text{qubit}} |\Phi^{(n)}\rangle \langle \Phi^{(n)}|$.
-

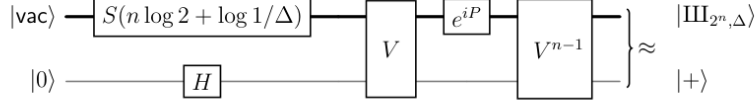


Figure 2: Text appearing in the image

Many figures consist of multiple images under a single caption. In these cases, the pipeline captures the entire group of images above the caption as a single image, since secondary images often provide context for understanding the quantum circuit. The full caption is then associated as the descriptive text for the combined image.

An additional exceptional case occurs when non-caption descriptive text ends with a period (.). The `CAPTION_REGEX` may mistakenly identify this as a caption, resulting in a cropped image that contains only text. Such cases are rare but represent one of the limitations of the current approach.

3 Data Filtering

Not all papers in the quantum physics domain contain quantum circuit images, so it is necessary to classify the extracted image-text pairs as either quantum circuit or not quantum circuit. An effective filtering approach must consider the two main objectives of the dataset:

Since the dataset is intended to train an Image-to-Text model, ideally every entry should describe a quantum circuit. Therefore, the classifier should minimize False Positives to ensure high accuracy of the dataset.

The dataset should be compiled from as few papers as possible, starting sequentially from the beginning of the list. This means it is also important to minimize False Negatives so that no quantum circuit image is missed.

It is not possible to completely eliminate both False Positives and False Negatives simultaneously. In this solution, priority is given to reducing False Positives to create a highly accurate dataset, which is crucial for training a reliable model. However, efforts are still made to retain as many True Positives as possible, so that False Negatives are

not entirely disregarded.

3.1 Text Based Classification

The solution uses regex-based heuristics to classify image captions as either quantum circuit or not quantum circuit. The underlying idea is that an image caption typically provides a direct description of the corresponding image and often contains specific keywords—referred to as decider words—that indicate whether the image depicts a quantum circuit. Other descriptive texts related to the image may not consistently include such keywords. Therefore, the main goal of this approach is to design a regex pattern that can effectively detect these decider words in captions and classify the images accurately.

3.2 Regex-Based Classifier

The goal of the regex-based classifier is to define two sets of keywords for classifying captions:

Positive words – Words that, when present in a caption, indicate a high likelihood that the image depicts a quantum circuit.

Negative words – Words that, when present in a caption, suggest that the image is unlikely to be a quantum circuit.

Both sets are carefully chosen to maximize classification accuracy. The positive words help identify captions with a high probability of representing quantum circuits, minimizing False Negatives, while the negative words help filter out unlikely captions, reducing False Positives. To achieve this, the word patterns of existing captions are analyzed, and a regex pattern is constructed to automatically classify each caption based on the presence of these keywords. To refer to the workings of this function, refer to `check_quantum_caption()` function in `main.py`.

3.3 Regex Construction using Tf-Idf

Manually selecting keywords from captions to construct a regex is possible, but this approach is neither efficient nor scalable for large or new sets of quantum physics papers. To ensure reliability and scalability, the solution analyzes word patterns programmatically using a dataset of 1,000 papers collected in two ways:

Randomly selecting 1,000 PDFs from indices 20,000 to 26,000 in the paper list.

Sequentially selecting 1,000 PDFs from indices 25,000 to 26,000.

Although both approaches aim to gather 1,000 papers, using two methods provides a broader perspective and enhances the generalizability of the regex. Papers were selected from the end of the list to assume that most quantum circuit images would already appear in the first 20,000 papers. This resulted in two datasets: one for analyzing word patterns (training dataset) and one for testing the patterns (testing dataset).

Using domain knowledge, a preliminary set of positive and negative words was constructed. The positive word included the phrase "quantum circuit", while negative words included "graph", "plots", and "tables". Applying this initial regex on the training dataset identified a small set of positive samples (few False Positives) and a large set of negative samples. However, this simple approach introduced many False Negatives because captions lacking the explicit positive phrase were missed.

To optimize the regex, the solution employed TF-IDF scoring. TF-IDF calculates the importance of each word within a document relative to its rarity across the corpus. For each word in the positive and negative datasets, the TF-IDF scores were summed across all documents, giving a measure of how important and representative each word is for its class. Figure 3 (a) and (b) show the summed Tf-Idf scores for Positive and Negative Sample set respectively. Words with higher scores are frequent in their class captions and distinctive to that class, making them ideal candidates for inclusion in the regex.

tfidf_df.head(50)		
	word	tfidf_score
1511	quantum	16.278049
313	circuit	16.040602
819	gate	13.834636
1513	qubit	13.419992
820	gates	12.154694
315	circuits	8.013252
1516	qubits	7.522458
1799	state	6.766883
925	implementation	5.209024
864	hadamard	5.079933
1099	logical	4.984846
1654	rotation	4.690738

tfidf_negative_df.head(50)		
...	12776	time 79.660933
	7661	line 78.579234
	10387	qubit 77.321178
	4250	different 73.329077
	12089	states 68.417503
	2467	blue 66.785809
	4951	energy 65.898337
	10633	red 65.197305
	10390	qubits 63.993712
	7668	lines 63.808719
	5042	eq 62.091729
	8874	number 61.895986
	8319	model 60.906738
	5098	error 60.815573
	3896	dashed 60.010516

Figure 3: (a) Summed Tf-Idf Scores of each word in positive sample set (b) Summed Tf-Idf Scores of each word in negative sample set

Some manual curation was still necessary, as certain words (e.g., "states") were dominant in both classes and could not reliably indicate one class. Unlike simple word frequency counts, TF-IDF highlights words that are characteristic of a particular sub-domain. For example, words like "photon", "magnetic", "electricity", or "resistance" indicate other sub-domains and reduce the likelihood that a caption belongs to a quantum circuit figure. By combining TF-IDF analysis with domain knowledge, the final regex captures dominant, class-specific words, improving both accuracy and generalizability.

You can also calculate TF-IDF scores for bigrams for both classes. However, including them in the regular expressions lead to loss of scalability, as the phrases are too specific. To see the exact workings for this, please refer to `get_important_words_list()` function in `regex_construction.py`.

4 Remaining Metadata Extaction

Metadata like the gates from the circuit and the quantum problem described by the circuit were comparatively harder to extract, due to their rare availability in the papers.

4.1 Quantum Gates Extraction

To identify the quantum gates used in a circuit, regular expressions were applied to the caption and other descriptive text associated with each image. The regex, generated with the help of ChatGPT, was designed to capture all possible variations in which a gate might be mentioned. You can refer to `get_quantum_gates()` function in `main.py`.

Although extracting gates directly from the circuit images using OCR could also be an approach, this approach has significant limitations. Many gates share the same symbolic representation, but their arrangement within the circuit conveys crucial information about the computation. OCR would detect the letters but fail to capture the arrangement and relationships between gates, leading to numerous errors. Therefore, text-based extraction using regex was chosen as a more accurate and reliable method.

4.2 Quantum Problem/Algorithm Extraction

The problem statement requires identifying the quantum problem that is being solved by the "gate". While quantum gates are the building blocks of a circuit, the actual problem is determined by the overall design of the circuit. The gates are just mere operators. However, most papers do not explicitly mention standard quantum algorithms like Shor's algorithm or Grover's algorithm. In fact, many quantum circuits are generic illustrations used to explain concepts rather than solve a specific algorithmic problem.

To capture the tiny number of occurrences of these standard algorithms, a regular expression containing the names of known quantum problems and algorithms was created (generated with the help of ChatGPT). This allows the system to identify any direct mentions of standard algorithms in captions or text.

For circuits that do not explicitly reference a known algorithm, a fallback approach is applied. Since captions often describe the function of a quantum circuit, we can infer some information about the problem from those captions. The solution uses "Noun Phrase Chunking" [1] to break captions into constituent noun phrases. You can refer the `get_quantum_problem()` function in `main.py`. The first three to four noun phrases are then combined using some hardcoded stopwords, to form a small coherent summary sentence, which approximates the problem the quantum circuit is addressing. This ensures

that every entry in the dataset contains at least some descriptive information about the circuit's purpose.

5 Final Pipeline Integration

The caption classifier and metadata extraction techniques were combined into a final data extraction pipeline. This integrated pipeline processes each image and caption in real time, classifying them as either quantum circuit or not quantum circuit on the fly. Once an image is classified as a positive example (i.e., a quantum circuit), the metadata extraction techniques are applied to gather additional information, such as descriptive texts, figure numbers, problem descriptions, and gate information. This approach ensures that only relevant images undergo detailed metadata processing, improving both efficiency and accuracy.

6 Result

The `dataset.json` file contains 250 image samples along with their associated metadata. These images were extracted from the first 854 papers in the dataset. Manual review determined a False Positive Rate of 26 percent, indicating that roughly one in four images was incorrectly classified as a quantum circuit.

Additionally, 117 out of 250 images (approximately 46 percent) did not have any quantum gates identified, meaning nearly half of the dataset lacks gate-level information. This highlights certain limitations in gate extraction and underscores the challenges of fully automated metadata collection for quantum circuits.

7 Source Code Files

`main.py` - Python file with the final pipeline that fetches quantum circuit images.

`caption_fetching.py` - Python file to fetch captions from the end of the list to create a json document, which acts as a dataset to construct the regular expression.

`regex_construction.py` - Python file to see the view the important words from both positive and negative dataset samples.

In order to verify the regex construction approach, you first need to run `caption_fetching.py` to collect the dataset in a json document and then run `regex_construction.py` to see the word importance scores using the json dataset.

Before running any file, install the modules listed in `requirements.txt` file in your environment.

8 Conclusion

This report presented an end-to-end data extraction pipeline for building a structured dataset of quantum circuit images and their corresponding textual metadata from research papers. By relying on PDF-based processing, spatial analysis, NLP techniques, and regex-based heuristics, the solution effectively identifies, extracts, and filters relevant images and descriptions suitable for training an Image-to-Text model. While the pipeline prioritizes reducing False Positives to ensure dataset quality, the evaluation results also highlight existing challenges, such as incomplete gate extraction and classification errors. Overall, the proposed approach provides a scalable and practical foundation for automated dataset creation, with clear scope for further refinement and improvement.

Bibliography

- [1] Data Knows All, “What is a noun phrase?.” <https://dataknowsall.com/blog/nounphrase.html>, 2024. Accessed: 22 December 2025.